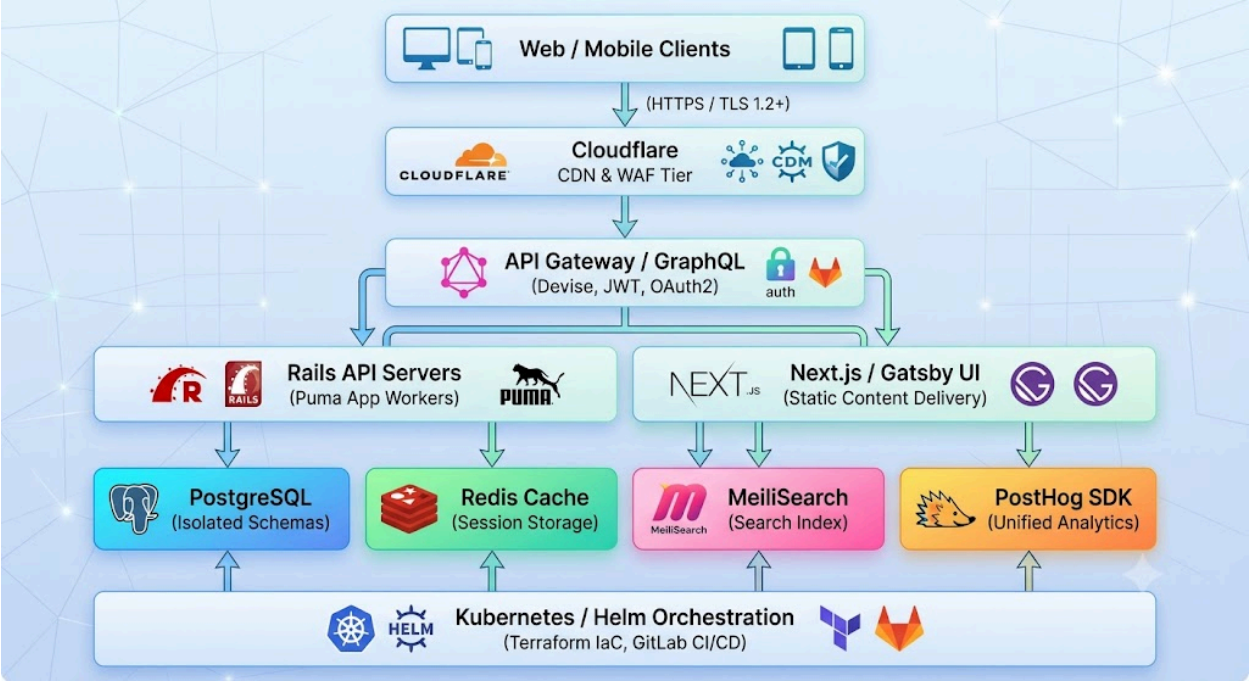
# LMS - Technical Acquisition & Due Diligence Audit

# Technical Acquisition & Due Diligence Audit

## 1. Platform Foundation & Technical Stack Overview

This system is built as a multi-tenant platform designed to serve student, teacher, and administrative workloads across multiple school districts and institutions. To evaluate the platform's portability, we must first map its baseline ecosystem as extracted from the system documentation.




### The Access & Edge Layer

All incoming client traffic (Web Browsers and Mobile Applications) passes through a **Cloudflare CDN and Web Application Firewall (WAF)** over secure TLS 1.2+. This layer handles edge caching, static asset delivery, and initial rate limiting.

Authentication is managed via an **API Gateway** deploying **Ruby on Rails' Devise framework**, utilizing **JSON Web Tokens (JWT)** and **OAuth2/Role-Based Access Control (RBAC)** to govern session lifetimes and user privileges. Real-world user behavior and system usage metrics are tracked via the **PostHog SDK**.

### The Application & Business Logic Layer

The core business logic is split between two frontend frameworks and a central backend API:

- **The Frontend Services:** Built using **Next.js** for dynamic user interactions (such as taking assessments, monitoring course progression, and administrative configuration) and **Gatsby** for high-performance marketing and informational landing pages.
- **The Backend Services:** Driven by a **Ruby on Rails** API framework running on the **Puma** web server inside containerized environments. It handles heavy-duty services such as the *Assessment Engine*, *Course & Content Delivery*, *Financial/Transaction Management*, and *Certification Workflows*.

### The Data & Storage Layer

- **PostgreSQL:** The primary relational database engine. To keep school districts isolated from one another, the system uses a **logical multi-tenancy model** where each institution operates inside its own isolated database schema.
- **Redis:** Serves as the high-speed caching and session-state management tier, preserving user login states and rapid-access system data.
- **MeiliSearch:** Powers the platform's localized search capabilities, serving fast course, user, and content index queries.
- **Amazon S3:** Stores all media assets, course content uploads, and digital certificates, protected by AES-256 at-rest encryption.

### Platform Architecture & DevOps

The entire infrastructure is modeled as **Infrastructure as Code (IaC) using Terraform**. Applications are containerized via **Docker**, orchestrated using **Kubernetes**, and deployed using **Helm charts** through automated **GitLab CI/CD pipelines**.

# Executive Summary

Our objective is to ensure that post-transaction, our internal engineering team can independently run, maintain, scale, and secure this platform without any ongoing financial or operational dependence on the vendor's engineering staff.

This report establishes our assessment of their current technology stack, followed by a prioritized engineering interrogation risk matrix. We have structured these risks from basic operational definitions to advanced failure modes. Each section contains a non-technical context block to outline the business and financial implications of technical failures to non-technical stakeholders.

# 2. Technical Risk Progression Matrices

Our team has organized the technical risks into four core pillars, progressing from basic assumptions to advanced systemic failure states.

## Pillar 1: Concurrency, Load Spikes, and Resource Exhaustion

**Non-Technical Business Context:**

When tens of thousands of users log in at the exact same moment (e.g., a synchronized state-wide certification exam or a high-traffic results announcement), the platform experiences a massive power surge. If the system's software configurations are not perfectly aligned with the hardware limits, the application will drop student connections, lose active exam progress, and suffer sudden downtime, resulting in contractual SLA penalties and brand damage.

**Risk & Question Progression**

- **Level 1 (Basic Baseline):** What is the verified maximum number of concurrent requests a single Puma application pod can process before server response times exceed our 3-second SLA?
- **Level 2 (Intermediate Mechanics):** How are API rate limits synchronized between Cloudflare's edge layer and the internal API Gateway to prevent legitimate students from being blocked during high-volume registration periods?
- **Level 3 (Advanced Failure Modes):** Under maximum concurrent load, what explicit guards prevent database connection pool starvation within the Rails-to-PostgreSQL layer when context switching across hundreds of isolated tenant schemas? If Redis memory limits are breached during a concurrent exam, what steps does the system take to prevent dropping active user sessions?

**Audit & Verification Requirements**

| Artifacts to Request | Live KT Demonstrations | Required Proof of Compliance | Impact of Missing Verification |
|---|---|---|---|
| • Production puma.rb configs.<br><br>• PostgreSQL connection pool configurations. | A live-monitored stress test simulating 1.5x peak concurrent traffic against a mock testing environment. | Grafana/CloudWatch telemetry proving stable CPU utilization, clean connection re-use, and zero dropped HTTP requests. | Sudden, unresolvable platform crashes during critical high-stakes exam windows post-acquisition. |

| | | | |
|---|---|---|---|
| • Kubernetes Horizontal Pod Autoscaler (HPA) rules. | | | |

## Pillar 2: Tenant Isolation and Cross-Data Contamination

**Non-Technical Business Context:**

Because multiple client institutions (e.g., competing schools, corporations, or government segments) share the same cloud servers, the software must maintain absolute separation between them. Think of it like a secure apartment building: everyone shares the framework, but no tenant should ever have a key to someone else's apartment. If the vendor's code has a hidden flaw, a user from Client A could inadvertently view, edit, or delete private student data or financial records belonging to Client B.

**Risk & Question Progression**

- **Level 1 (Basic Baseline):** Where exactly in the middleware codebase does the system intercept an HTTP request and execute the PostgreSQL SET search_path command to route a user to their specific database schema?
- **Level 2 (Intermediate Mechanics):** How is tenant isolation strictly enforced beyond the relational database, specifically within shared MeiliSearch indices and Amazon S3 file storage folders?
- **Level 3 (Advanced Failure Modes):** How does the background worker pool (e.g., Sidekiq/async processing) prevent tenant context bleeding across shared execution threads? If a multi-tenant background migration job crashes midway through, how does the system guarantee the database thread is wiped clean before it picks up a job for a completely different client?

**Audit & Verification Requirements**

| Artifacts to Request | Live KT Demonstrations | Required Proof of Compliance | Impact of Missing Verification |
|---|---|---|---|
| | | | |

| | | | |
|---|---|---|---|
| • Context-switching middleware source code.<br><br>• S3 IAM policy configurations.<br><br>• Shared index segmentation code. | Live execution of overlapping, concurrent API requests across different client IDs while tracing database connection states. | Raw log data demonstrating that database queries remain securely locked within the tenant's schema boundary under heavy load. | Catastrophic data leaks, regulatory non-compliance fines, and immediate loss of enterprise customer trust. |

## Pillar 3: Pipeline Automation, Configuration Drift, and Portability

**Non-Technical Business Context:**

For our team to own this system, we must be able to build and run it entirely on our own cloud infrastructure. If the vendor's developers have spent years making manual adjustments, tweaks, and quick-fixes directly on the production servers without recording those changes in the official source code blueprints (Terraform/Helm), the software is "locked" to the vendor's environment. Without verifying this, we are purchasing an asset we cannot safely move, clone, or update.

**Risk & Question Progression**

- **Level 1 (Basic Baseline):** Can the complete application suite be built from scratch into raw Docker images using only the code repositories currently checked into their main Git branch?
- **Level 2 (Intermediate Mechanics):** How does the automated pipeline orchestrate database migrations across hundreds of separate PostgreSQL tenant schemas simultaneously without locking core tables or interrupting live users?
- **Level 3 (Advanced Failure Modes):** What environment variables, security tokens, or third-party integration keys are manually injected into production because they are omitted from the infrastructure-as-code files? Does running a terraform plan against their live cloud architecture yield zero changes, or is there significant undocumented configuration drift?

**Audit & Verification Requirements**

| Artifacts to Request | Live KT Demonstrations | Required Proof of Compliance | Impact of Missing Verification |
|---|---|---|---|
| • Raw .gitlab-ci.yml files.<br><br>• Helm charts (values.yaml).<br><br>• Master Terraform state files. | A complete "cold-start" deployment of the entire platform into a brand-new, isolated cloud account managed by our team. | A fully operational staging environment built with zero manual interventions, zero errors, and zero vendor code calls. | Permanent vendor lock-in; inability to update features, patch bugs, or migrate cloud providers independently. |

## Pillar 4: High Availability, Resilience, and Disaster Recovery

**Non-Technical Business Context:**

Cloud hardware components occasionally experience physical failures or localized power blackouts. The vendor's contract promises that if a major data center goes dark, the platform will recover fully in under 4 hours. However, if that recovery process relies on a checklist of manual steps known only to a few vendor engineers, we face severe business exposure if those individuals leave the company or become unavailable.

**Risk & Question Progression**

- **Level 1 (Basic Baseline):** Where are the system's database backups stored, how frequently are they validated, and where are the corresponding decryption keys kept?
- **Level 2 (Intermediate Mechanics):** What happens to active student connections and ongoing exam states when a single Kubernetes node or cloud availability zone undergoes an abrupt hardware failure?
- **Level 3 (Advanced Failure Modes):** What is the exact sequence of manual vs. automated steps required to fail over the entire platform to a secondary cloud region? How does the architecture handle global DNS propagation latency and SSL certificate re-issuance within the 4-hour Recovery Time Objective (RTO)?

**Audit & Verification Requirements**

| Artifacts to Request | Live KT Demonstrations | Required Proof of Compliance | Impact of Missing Verification |
|---|---|---|---|
| • Disaster Recovery (DR) playbook.<br><br>• Database replication scripts.<br><br>• Backup integrity logs. | A live, unannounced termination of the primary database cluster node during active automated traffic simulation. | Infrastructure metrics confirming automatic failover to the secondary node within the contractually mandated SLA limits. | Multi-day platform outages, data loss, and breach of core enterprise customer contracts during a cloud failure. |

# 3. High-Value Handover Interview Questions

The acquisition team will use the following targeted interview questions during live knowledge transfer sessions to extract unwritten tribal knowledge.

## Uncovering Tribal Knowledge

1. Which specific production outages or high-severity incidents over the past 24 months required manual intervention from your principal developers because standard alerting tools failed to identify the root cause?
2. What operational routines, cache-clearing practices, or database index re-indexing operations are regularly performed manually by your team but remain undocumented in your standard operations guides?
3. If your lead platform engineer were unavailable for an extended period, which specific sub-systems or integrations (e.g., GeM, NSDL, Razorpay modules) would present the highest risk to modify?

### Assessing Technical Debt & Code Health

4. Which modules within the Rails API or Next.js codebases currently have the highest complexity scores, or are considered too fragile to refactor easily?
5. What are the specific dependencies or structural limitations that have prevented your team from updating your core frameworks (Ruby on Rails, Next.js) to their latest stable security releases?
6. Are there any custom code workarounds or direct database hotfixes currently running in the production cluster that have not been committed back to the main Git repository?

### Identifying Latent Business Risks

7. What is the precise cloud infrastructure cost breakdown per active tenant schema, and what are the primary cost drivers as data storage and user volumes increase?
8. Are there any open-source components, third-party libraries, or custom search plugins utilized within this platform that require commercial enterprise licensing if redistributed under our corporate ownership?
9. What known architectural constraints limit the platform's ability to easily decouple single features—such as offering the SMS or WiseBPM modules as standalone solutions?

# 4. Final Acquisition Acceptance Criteria

Prior to closing the transaction and executing the final contract sign-off, the acquisition engineering team requires verified confirmation of the following conditions:

| ACQUISITION TECHNICAL COMPLIANCE SIGN-OFF |
|---|
| [ ] REQUIREMENT 1: Independent Cold-Start Build & Deploy Verified |
| [ ] REQUIREMENT 2: Zero Vendor-Owned Accounts or Commercial Keys |
| [ ] REQUIREMENT 3: 100% Code Coverage & Schema Portability Proven |
| [ ] REQUIREMENT 4: Automated Failover & Disaster Runbooks Executed |
| [ ] REQUIREMENT 5: Full Disclosure of Production Incident History |

- **Requirement 1:** Our internal engineering team must execute a successful, automated cold-start deployment of the entire LMS ecosystem into our corporate AWS infrastructure with zero operational assistance from the vendor.

- **Requirement 2:** The vendor must demonstrate that all third-party integrations (e.g., Postmark, Cloudflare) can be seamlessly transitioned to our corporate billing credentials without requiring code alterations or database schema patches.
- **Requirement 3:** A complete automated testing suite execution must yield a passing rate that satisfies our corporate software quality standards, with explicit validation of the multi-tenant schema isolation logic.
- **Requirement 4:** The vendor must supply complete, unredacted post-mortem reports for all high-severity platform incidents spanning the previous 24 months.
- **Requirement 5:** The vendor must sign off on a comprehensive Third-Party Dependency Register confirming that no open-source components violate our corporate compliance or IP distribution policies.

# Handover checklist

# Handover Checklist:

## 1. Codebases, Version Control, and Repositories

We must secure **Root Administrative Ownership** (not just write/developer access) of the master Git repositories. The delivery must include complete, unencrypted commit histories and build configurations.

- **[ ] Core Application Source Code:**
  - **Backend API Service Repo:** Complete Ruby on Rails source code, including custom assessment engines, grading logic, and certification workflow controllers.
  - **Dynamic Frontend UI Repo:** Next.js application codebase governing the active student, teacher, and administrative portal views.
  - **Static Web Frontend Repo:** Gatsby source code governing marketing, landing, and public informational pages.
  - **Mobile App Repos:** Full native or cross-platform source files for iOS and Android mobile learning applications (if packaged).
- **[ ] Infrastructure as Code (IaC) & Cloud Orchestration:**
  - **Terraform Repository:** Complete IaC scripts mapping cloud network topologies, VPCs, subnets, and Kubernetes cluster definitions.
  - **Kubernetes & Helm Configurations:** Raw Dockerfiles, active Kubernetes deployment manifests, and Helm charts (`values.yaml`) used to orchestrate container nodes.
  - **CI/CD Pipeline Frameworks:** Complete `.gitlab-ci.yml` or GitHub Actions workflow configurations defining automated build, test, and container packaging sequences.
- **[ ] Database and Search Indices:**
  - **PostgreSQL Migration Ledger:** Sequential database schema and schema-isolation migration files used to construct the multi-tenant relational data structures.
  - **MeiliSearch Re-indexing Assets:** Custom indexing configuration files, data schemas, and synchronization worker scripts utilized by MeiliSearch.
  - **Cache Strategy Scripts:** Initialization and memory-eviction management configurations for the Redis caching layers.

## 2. Technical, Architectural, and Blueprint Documentation

The following documentation must be delivered in fully editable, structured formats to allow our engineering team to manage and scale the platform.

- **[ ] Engineering & Solution Designs:**
  - High-Level Design (HLD) and Low-Level Design (LLD) documents detailing application logic.
  - Detailed architectural diagrams across five critical perspectives: Logical Architecture, Physical Deployment, Network Topologies, Infrastructure Topology, and Component Interaction.

    - Database Schema Design documentation, Entity-Relationship (ER) Diagrams, and a comprehensive Data Dictionary.
- **[ ] Operational Playbooks & Manuals:**
    - **System Administration & Configuration Guides:** Master runbooks for maintaining application-wide settings and system variables.
    - **Platform Tenant Onboarding Guide:** Repeatable technical procedures for provisioning a new institution, mapping custom domains, and applying white-label theme sets.
    - **Platform Runbooks:** Troubleshooting guides mapping every production logging exception or system alert directly to a manual or automated resolution path.
    - **Disaster Recovery (DR) & Business Continuity Plans:** Step-by-step procedures detailing regional database failover paths, cryptographic key restorations, and automated verification of daily cold backups.
- **[ ] Interfaces & Compliance Registers:**
    - **API & Integration Documentation:** Open API/Swagger specifications for all internal services, external webhooks, and third-party API configurations.
    - **Third-Party Dependency Register:** A complete inventory of all open-source libraries, frameworks, gems, and commercial components used across the stack.
    - **Open Source License Compliance Audit:** Verification proving that no open-source or copyleft frameworks compromise our corporate IP or restrict commercial redistribution.

## 3. Security, Quality Assurance, and Audit Trail Materials

We must demand complete visibility into the system's current security posture and operational baggage.

- **[ ] Vulnerability & Assessment Records:**
    - The most recent external Vulnerability Assessment and Penetration Testing (VAPT) and security assessment reports.
    - Unredacted incident logs and technical root-cause post-mortems for all high-severity (P1/P2) platform outages from the past 24 months.
- **[ ] Access and Control Frameworks:**
    - Security Controls Matrix defining Role-Based Access Controls (RBAC) across student, teacher, system administrator, and super-admin personas.
    - Cryptographic Governance Plan detailing JWT signing key generation, storage locations, validation lifecycles, and user-token revocation logic.

## 4. Operational Assets, Inventories, and Deployment Secrets

The practical "keys to the digital kingdom" required to keep the production platform live without interruption.

- **[ ] SSL Certificates & DNS Assets:**
    - Active SSL/TLS Certificate inventory, root certificates, and associated private keys for all core domains and subdomains.
    - DNS Zone Files and complete structural registers detailing all active domain names, registrars, and forwarding rules.
- **[ ] Environmental Variables & Identity Assets:**

    - Production Environment Inventory mapping every runtime environment variable, hidden app config flag, and operational endpoint.
    - Master passwords, database seed credentials, administrative connection strings, and secure SSH keys required to access backend servers.

## 5. Third-Party Corporate Vendor & SaaS Accounts

To prevent immediate service cutoffs, master billing administrative ownership of these critical operational vendor accounts must be signed over directly to our corporate corporate development entity.

- **[ ] Primary Cloud Infrastructure Workspace:** Master administrative root access to the active hosting environment (e.g., **AWS root accounts** hosting EKS, RDS PostgreSQL databases, Redis nodes, and S3 media storage networks).
- **[ ] Edge Defense & Content Delivery Network:** The corporate **Cloudflare account** managing active DNS zones, Web Application Firewall (WAF) rate-limiting rules, and global asset caches.
- **[ ] Transactional Communication Gates:** Master accounts managing automated communication routes (e.g., **Postmark** or transactional SMS relays) responsible for system verifications, student signup tokens, and critical app alerts.
- **[ ] Analytics and Platform Telemetry Hubs:** Root access to administrative monitoring and user-behavior platforms, specifically the live enterprise **PostHog workspace**.